Rozszerzenie A — wizualizacja nagrań i współczynników MFCC

Cel: przedstawienie sygnału mowy w postaci wykresu amplitudy oraz mapy współczynników MFCC.

Polecenia:

1. Utworzyć plik `visualize_audio.py`.
2. Wczytać nagranie WAV o częstotliwości próbkowania 16 kHz.
3. Wygenerować wykres przebiegu czasowego sygnału.
4. Wyznaczyć i zwizualizować współczynniki MFCC.
5. Porównać nagrania tego samego słowa wykonane przez dwie różne osoby.
6. Zbadać wpływ liczby współczynników MFCC: `13`, `20` i `40`.

Przykładowy kod pomocniczy:

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

PATH = "data/anna/start/000.wav"  # Ustawić ścieżkę do istniejącego nagrania.
signal, sr = librosa.load(PATH, sr=16000, mono=True)

mfcc = librosa.feature.mfcc(
    y=signal,
    sr=sr,
    n_mfcc=20,
    n_fft=400,
    hop_length=160
)

fig, axes = plt.subplots(2, 1, figsize=(11, 7))
time = np.arange(len(signal)) / sr
axes[0].plot(time, signal)
axes[0].set(title="Sygnał audio", xlabel="Czas [s]", ylabel="Amplituda")

img = librosa.display.specshow(
    mfcc, x_axis="time", sr=sr, hop_length=160, ax=axes[1]
)
axes[1].set(title="Współczynniki MFCC")
fig.colorbar(img, ax=axes[1])
plt.tight_layout()
plt.show()
```

W razie braku biblioteki `matplotlib` wykonać `python -m pip install matplotlib`, a następnie dodać tę zależność do `requirements.txt`.

Wymagany rezultat: co najmniej dwa zapisane wykresy, porównanie różnych liczby MFCC oraz trzy wnioski z obserwacji.

Zagadnienie do wyjaśnienia: ograniczenia rozpoznawania słów wyłącznie na podstawie amplitudy sygnału.

Rozszerzenie B — analiza jakości zbioru danych

Cel: wykrycie błędów i nierównowagi w zbiorze nagrań.

Polecenia:

1. Utworzyć plik `dataset_report.py`.
2. Policzyć nagrania osobno dla każdego mówcy i każdej komendy.
3. Sprawdzić czas trwania oraz częstotliwość próbkowania nagrań.
4. Wykryć nagrania puste, uszkodzone i podejrzanie krótkie.
5. Wskazać najmniej oraz najliczniej reprezentowane klasy.
6. Zaproponować sposób uzupełnienia zbioru przy niezrównoważonej liczbie przykładów.

Fragment kodu:

```
from collections import Counter
from pathlib import Path
import soundfile as sf

root = Path("data")
counts = Counter()

for path in root.glob("*/*/*.wav"):
    speaker, word = path.parts[-3], path.parts[-2]
    try:
        info = sf.info(path)
    except RuntimeError:
        print("Plik uszkodzony:", path)
        continue

    counts[(speaker, word)] += 1
    if info.samplerate != 16000:
        print("Nieprawidłowa częstotliwość:", path, info.samplerate)
    if info.duration < 0.3:
        print("Bardzo krótkie nagranie:", path)
```

```
for item, count in sorted(counts.items()):
    print(item, count)
```

Wymagany rezultat: raport zawierający liczbę nagrań, podstawowe statystyki czasu trwania, wskazanie nieprawidłowych plików oraz krótką ocenę jakości zbioru.

Uwaga: nie usuwać trudnych, ale poprawnych nagrań wyłącznie w celu zwiększenia skuteczności klasyfikacji.

---

5. Rozszerzenie C — augmentacja nagrań

Cel: porównanie procesu treningu z oryginalnymi danymi i danymi sztucznie rozszerzonymi.

Polecenia:

1. Rozdzielić funkcję `features(path)` na dwa etapy: `load_audio(path)` oraz `audio_to_features(audio)`.
2. Zaimplementować dodawanie szumu losowego do sygnału.
3. Dodać co najmniej jedną z metod: modyfikację tempa, zmianę wysokości głosu lub przesunięcie w czasie.
4. Wykonywać augmentację **dopiero po podziale danych i wyłącznie na zbiorze treningowym**.
5. Dostosować długość sygnału po każdej transformacji do `N_SAMPLES`.
6. Porównać wynik modelu trenowanego bez augmentacji i z augmentacją.

Przykład dodawania szumu:

```
import numpy as np

def add_noise(audio, level=0.01, rng=None):
    if rng is None:
        rng = np.random.default_rng(42)
    noisy = audio + rng.normal(0, level, size=audio.shape)
    return np.clip(noisy, -1.0, 1.0).astype(np.float32)
```

Przykłady pozostałych transformacji:

```
# Zmiana tempa bez zmiany wysokości głosu.
faster = librosa.effects.time_stretch(audio, rate=1.1)
faster = librosa.util.fix_length(faster, size=N_SAMPLES)

# Zmiana wysokości głosu bez zmiany czasu trwania.
higher = librosa.effects.pitch_shift(audio, sr=SR, n_steps=2)
```

Istotne wymaganie: funkcja `load_dataset()` w projekcie bazowym oblicza MFCC przed podziałem danych. Przy implementacji augmentacji należy najpierw podzielić **ścieżki do**

nagrań według mówców, a dopiero później odczytywać nagrania treningowe i generować ich modyfikacje. Oryginał i jego kopie nie mogą trafić do różnych zbiorów.

Wymagany rezultat: implementacja minimum dwóch metod augmentacji, przykłady nagrań po zmianach, tabela porównawcza wyników oraz wnioski.

Rozszerzenie D — modyfikacja architektury CNN

Cel: zbadanie wpływu liczby warstw i parametrów sieci na jakość klasyfikacji.

Polecenia:

1. Przenieść tworzenie sieci z funkcji `train()` do nowej funkcji `build_model(variant, n_classes)`.
2. Zachować podstawową wersję sieci jako wariant `base`.
3. Utworzyć wariant `extended` z dodatkową warstwą konwolucyjną oraz warstwą `Dropout`.
4. Wykorzystywać tę samą funkcję budowania modelu podczas treningu i predykcji.
5. Zmierzyć liczbę parametrów obu architektur oraz czas treningu.
6. Porównać skuteczność wariantów przy identycznym podziale danych.

Przykładowa implementacja:

```
from torch import nn

def build_model(variant="base", n_classes=5):
    if variant == "extended":
        return nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten(),
            nn.Dropout(0.3), nn.Linear(64, n_classes)
        )

    return nn.Sequential(
        nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
        nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten(),
        nn.Linear(32, n_classes)
    )
```

Przykładowy pomiar liczby parametrów:

```
parameter_count = sum(p.numel() for p in model.parameters())
print("Liczba parametrów:", parameter_count)
```

Zapisywanie konfiguracji: wraz z wagami modelu należy zapisać nazwę wariantu i niezbędne parametry przetwarzania. Przykładowy fragment checkpointu:

```
torch.save({
    "weights": model.state_dict(),
    "mean": torch.tensor(mean),
    "std": torch.tensor(std),
    "variant": variant,
    "words": WORDS,
    "n_mfcc": n_mfcc,
}, model_path)
```

Podczas predykcji odczytać `variant` oraz `n_mfcc` i odtworzyć właściwą architekturę przed wywołaniem `load_state_dict()`.

Wymagany rezultat: porównanie co najmniej dwóch architektur z podaniem liczby parametrów, czasu treningu i metryk jakości.

Rozszerzenie E — walidacja, metryki i analiza błędów

Cel: przygotowanie poprawnej procedury oceny modelu.

Polecenia:

1. Rozszerzyć podział `train/test` do `train/validation/test`.
2. Zapewnić brak wspólnych mówców pomiędzy zbiorami, o ile pozwala na to liczba uczestników.
3. Obliczać średnią stratę dla całej epoki, a nie wyłącznie ostatniej partii.
4. Zapisywać wyniki każdej epoki w historii treningu.
5. Wybierać hiperparametry i najlepszą epokę wyłącznie na podstawie walidacji.
6. Na zakończenie przeprowadzić pojedynczą ocenę wybranego modelu na zbiorze testowym.
7. Obliczyć accuracy, precision, recall, F1-score oraz macierz pomyłek.
8. Przeanalizować minimum pięć niepoprawnie sklasyfikowanych nagrań.

Schemat gromadzenia historii:

```
history = []

for epoch in range(epochs):
    # Wykonać trening na train.
    # Obliczyć średnią stratę dla wszystkich próbek treningowych.
    # Ocenić model na validation.
```

```
    history.append({
        "epoch": epoch + 1,
        "train_loss": float(train_loss),
        "val_accuracy": float(val_accuracy),
    })
```

`train_loss` oraz `val_accuracy` wymagają samodzielnego obliczenia w pętli. Przy liczeniu średniej straty z partii zastosować ważenie liczbą przykładów w partii.

Przykład wyświetlenia metryk:

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(
    y_true, y_pred,
    labels=list(range(len(WORDS))),
    target_names=WORDS,
    zero_division=0
))
print(confusion_matrix(
    y_true, y_pred,
    labels=list(range(len(WORDS)))
))
```

Wartości `y_true` i `y_pred` powinny pochodzić z niezależnego zbioru testowego.

Wymagany rezultat: wykres przebiegu treningu, macierz pomyłek, raport metryk oraz analiza błędów.

---

8. Rozszerzenie F — parametry treningu w menu

Cel: umożliwienie eksperymentowania bez ręcznej edycji wartości zapisanych wewnątrz funkcji.

Polecenia:

1. Rozszerzyć opcję treningu w pliku `main.py`.
2. Dodać wprowadzanie współczynnika uczenia (`learning rate`), rozmiaru partii (`batch size`) i wariantu architektury.
3. Umożliwić zmianę liczby współczynników MFCC.
4. Przekazywać wartości jako argumenty funkcji `train()`.
5. Wprowadzić sprawdzanie poprawności danych wpisywanych przez użytkownika.
6. Zapisywać parametry użyte przy trenowaniu wraz z modelem.
7. Zagwarantować identyczne ustawienia przetwarzania audio podczas treningu i rozpoznawania.

Przykładowe wartości do eksperymentów:

Parametr	Warianty
Learning rate	`0.0001`, `0.001`, `0.01`
Batch size	`8`, `16`, `32`
Liczba MFCC	`13`, `20`, `40`
Architektura	`base`, `extended`

Wymagany rezultat: nowe opcje konfiguracji w menu i zapis parametrów modelu.

Rozszerzenie G — rozpoznawanie bezpośrednio po nagraniu

Cel: połączenie rejestracji głosu z natychmiastowym wykorzystaniem wytrenowanej sieci.

Polecenia:

1. Dodać w `main.py` opcję `5 - Nagraj i rozpoznaj`.
2. Zarejestrować nagranie o długości `DURATION` przy pomocy `sounddevice`.
3. Zapisać je jako plik WAV przy pomocy `soundfile`.
4. Przekazać zapisany plik do istniejącej funkcji `predict()`.
5. Wyświetlić rozpoznaną klasę oraz wyniki dla wszystkich komend.
6. Dodać możliwość odrzucenia klasyfikacji poniżej ustalonego progu wyniku softmax.
7. Dobrać próg na zbiorze walidacyjnym i przetestować działanie z ciszą oraz nieznanymi słowami.

Uwaga metodyczna: wynik softmax nie stanowi gwarantowanego prawdopodobieństwa poprawności. Sam próg może nie wystarczyć do odrzucania nieznanych wypowiedzi. Rozszerzeniem zaawansowanym może być dodanie osobnych klas `cisza` i `inne`, wraz z ich przykładami treningowymi.

Wymagany rezultat: działająca opcja menu pozwalająca nagrać i sklasyfikować nową komendę.

Proponowany zestaw eksperymentów

Eksperyment	Architektura	MFCC	Learning rate	Batch	Augmentacja	Badany wpływ
1	base	20	0.001	16	Nie	Wynik bazowy
2	extended	20	0.001	16	Nie	Architektura
3	base	40	0.001	16	Nie	Liczba cech
4	base	20	0.0001	16	Nie	Współczynnik uczenia
5	base	20	0.001	16	Tak	Augmentacja

W każdym eksperymencie stosować ten sam podział mówców. Zapisywać ziarna losowości (`random`, NumPy, PyTorch), konfigurację, czas obliczeń i wyniki. W miarę dostępnych zasobów powtórzyć wybrane eksperymenty dla kilku ziaren losowości.

Minimalny zakres raportu

1. Opis przygotowanego zbioru danych i zastosowanego podziału.
2. Opis zaimplementowanych zmian w kodzie.
3. Wykresy sygnału i współczynników MFCC.
4. Zestawienie parametrów i wyników poszczególnych eksperymentów.
5. Wykresy treningu i macierz pomyłek.
6. Omówienie co najmniej pięciu błędów modelu.
7. Wnioski: które modyfikacje poprawiły wyniki, które okazały się nieskuteczne i dlaczego.

Kontrola poprawności i typowe problemy

- **Brak pliku WAV:** zweryfikować ścieżkę w katalogu `data/` oraz katalog roboczy konfiguracji Run w PyCharm.
- **Brak dostępu do mikrofonu:** sprawdzić uprawnienia Windows i domyślne urządzenie wejściowe. Dopuszczalne jest użycie wcześniej zarejestrowanych nagrań.
- **Błąd `Potrzeba co najmniej 3 mówców`:** zebrać dane dodatkowych osób; nie kopiować tych samych nagrań pod innymi identyfikatorami.
- **Błąd `load_state_dict`:** sprawdzić zgodność architektury zapisanej w checkpointcie z architekturą podczas predykcji.
- **Bardzo wysoka skuteczność:** wykluczyć przeciek danych oraz nadmierne podobieństwo nagrań pomiędzy zbiorami.
- **Zmiana `WORDS`:** dostarczyć dane dla nowej klasy, dopasować rozmiar warstwy wyjściowej i ponownie wytrenować sieć.
- **Zmiana MFCC:** zapisać konfigurację cech i stosować ją konsekwentnie podczas trenowania i predykcji.

- **Normalizacja:** statystyki `mean` i `std` wyznaczać wyłącznie ze zbioru treningowego.